

贪心算法

1. 分数背包问题

给定 n 个物品，物品价值分别为 P_i ，重量分别为 W_i ，背包容量为 M 。每种物品可部分装入，求最大化总价值 $\sum P_i X_i$ 且满足 $\sum W_i X_i \leq M$ 。

2. 带截止时间的作业调度

有 n 个作业需在一台机器上执行，每个作业单位时间完成，作业 i 有截止时间 d_i 和收益 p_i 。求最大收益。

3. 硬币找零问题

假设有数目不限的面值为 25 美分、10 美分、5 美分、1 美分的硬币，请使用最少个数的硬币凑出 3.33 美元。

4. 海岸线基站问题

海面上有一些船需与陆地通信，在 x 轴上布置基站覆盖半径为 d 的区域内的所有点。问在 x 轴上至少要安放几个点才能将所有船覆盖。

5. 生产与存储计划

工厂每月生产成本波动，存储成本固定为 1。已知 n 个月的生产成本 c_i 和需求量 y_i ，决策每月生产数量使总成本最低。

6. 黑白点连线

直线上 $2n$ 个点， n 黑 n 白，相邻点距离为 1，将每个白点与一黑点相连，使连线总长度最小。

7. 活动选择问题

活动集合 $S = \{1, 2, \dots, n\}$ ，活动 i 有起始时间 s_i 和终止时间 f_i ，同一时间只有一个活动使用资源。求 S 的最大相容子集。

8. 哈夫曼编码问题

给定字符集 $C = \{c_1, c_2, \dots, c_n\}$ 及其出现频率 $f = \{f_1, f_2, \dots, f_n\}$ ，构造 C 的最优前缀编码，使带权路径长度 $B(T) = \sum_{c \in C} f(c) \cdot d_T(c)$ 最小。

9. 最小生成树问题 (Kruskal 算法)

给定带权无向连通图 $G = (V, E)$ ，权函数 W ，求 G 的最小生成树，即权值 $W(T) = \sum_{(u,v) \in T} W(u, v)$ 最小的生成树。

10. 单源最短路径问题 (Dijkstra 算法)

给定带权图（每条边权值 ≥ 0 ）和源点 v ，求 v 到其他所有顶点的最短路径。

2 贪心算法解题的”固有方法论”

贪心算法 (Greedy Algorithm) 的核心思想是”只顾眼前，不顾未来”。在每一步选择中都采取在当前状态下最好 (局部最优) 的选择，从而希望导致结果是全局最优的。

2 解题四步法：

1. 贪心策略选择 (Greedy Choice):

思考：用什么标准来做”贪心”？

技巧：通常需要对数据进行排序或使用优先队列

2. 可行性检查 (Feasibility Check):

思考：这个选择是否满足限制条件？

3. 子问题化 (Sub-problem):

做完一个选择后，原问题变成了什么样？

4. 证明/验证 (Proof/Verify):

尝试举反例，如果举不出反例，通常策略是正确的
常见的证明方法是”交换论证法”

1. 分数背包问题

给定 n 个物品，物品价值分别为 $P_1, P_2, \dots, P_n$ ，物品重量分别为 $W_1, W_2, \dots, W_n$ ，背包容量为 M 。每种物品可部分装入到背包中。输出 $X_1, X_2, \dots, X_n$ ($0 \leq X_i \leq 1$)，使得 $\sum_{i=1}^n P_i X_i$ 最大，且 $\sum_{i=1}^n W_i X_i \leq M$ 。试设计一个算法求解该问题，分析算法的正确性。

- **贪心策略**：优先选择单位重量价值 ($v_i = P_i/W_i$) 最高的物品
- **时间复杂度**： $O(N \log N)$ (排序时间)

```
struct Item {
    int id;
    double p, w, ratio;
};

bool cmp(Item a, Item b) {
    return a.ratio > b.ratio;
}

void solve_knapsack(int n, double M,
                   vector<double> P,
                   vector<double> W) {
    vector<Item> items(n);
    for(int i=0; i<n; ++i)
        items[i] = {i, P[i], W[i], P[i]/W[i]};

    sort(items.begin(), items.end(), cmp);

    vector<double> X(n, 0.0);
    double totalVal = 0;

    for(int i=0; i<n; ++i) {
        if(items[i].w <= M) {
            X[items[i].id] = 1.0;
            totalVal += items[i].p;
            M -= items[i].w;
        } else {
            X[items[i].id] = M / items[i].w;
            totalVal += items[i].p * X[items[i].id];
            break;
        }
    }
}
```

解题步骤：

1. 计算每个物品的单位重量价值 $v_i = P_i/W_i$
2. 按单位价值从高到低排序所有物品
3. 依次选择物品，若能完整装入则全部拿走，否则拿部分装满背包
4. 累加总价值，输出每个物品的装入比例 X_i

2. 带截止时间的作业调度

有 n 个作业需要在一台机器上执行，一个时刻机器上只能执行一个作业，每个作业可在单位时间内完成，作业 i 有截止时间 d_i ，当作业 i 在截止时间被执行完，则可获得 p_i 的收益，请设计算法获得最大收益，并分析算法的正确性。

- **贪心策略**：优先选择收益 p_i 最大的作业
- **策略执行**：将作业按收益降序排序，对每个作业尝试将其安排在不超过截止时间的最晚空闲时间槽
- **时间复杂度**： $O(N^2)$ (简单查找) 或 $O(N \log N)$ (并查集优化)

```
struct Job {
    int id, d, p;
};

bool cmp(Job a, Job b) {
    return a.p > b.p;
}

int solve_jobs(int n, vector<Job>& jobs) {
    sort(jobs.begin(), jobs.end(), cmp);

    int max_deadline = 0;
    for(auto& j : jobs)
        max_deadline = max(max_deadline, j.d);

    vector<bool> slot(max_deadline + 1, false);
    int totalProfit = 0;

    for(int i=0; i<n; ++i) {
        for(int j = min(max_deadline, jobs[i].d);
            j >= 1; --j) {
            if(!slot[j]) {
                slot[j] = true;
                totalProfit += jobs[i].p;
                break;
            }
        }
    }
    return totalProfit;
}
```

解题步骤：

1. 按收益从高到低对所有作业排序
2. 找出最大截止时间，创建时间槽数组
3. 对每个作业，从其截止时间开始向前查找第一个空闲时间槽
4. 找到则占用该时间槽并累加收益，否则跳过该作业
5. 继续处理下一个作业，直到所有作业处理完毕

3. 硬币找零问题

假设有数目不限的面值为 25 美分、10 美分、5 美分、1 美分的硬币，请使用最少个数的硬币凑出 3.33 美元。

- **贪心策略：** 优先使用面值最大的硬币
- **注意事项：** 此策略对特定面值体系（如美分）有效，对任意面值未必有效（需用动态规划）
- **时间复杂度：** $O(K)$ ，K 为硬币种类数

```
void solve_coins(int cents) {
    int coins[] = {25, 10, 5, 1};
    int count = 0;

    for(int val : coins) {
        int num = cents / val;
        count += num;
        cents %= val;
        cout << "面值 " << val << ": "
              << num << " 个" << endl;
    }
    cout << "总个数: " << count << endl;
}
```

解题步骤：

1. 将硬币面值按从大到小排序
2. 从最大面值开始，计算当前金额能包含多少个该面值硬币
3. 用取模运算得到剩余金额
4. 对下一个面值重复上述过程，直到金额为 0
5. 累加所有硬币数量得到答案

示例：333 美分 = $13 \times 25 + 0 \times 10 + 1 \times 5 + 3 \times 1 = 17$ 个硬币

4. 海岸线基站问题

海面上有一些船需要与陆地进行通信，需要在海岸线上布置一些基站。现将问题抽象为，在 x 轴上方，给出 N 条船的坐标 $p_1, p_2, \dots, p_n$, $p_i = (x_i, y_i)$, $y_i \geq 0$, $x_i^2 + y_i^2 \leq d^2$, $1 \leq i \leq n$ 。在 x 轴上安置的基站可以覆盖半径为 d 的区域内的所有点，问在 x 轴上至少要安放几个点才可以将 x 轴上方的点都覆盖起来。试设计一个算法求解该问题，并分析算法的正确性。

- **问题转化：** 每个船 i 在 x 轴上的覆盖范围为区间 $[x_i - \sqrt{d^2 - y_i^2}, x_i + \sqrt{d^2 - y_i^2}]$ ，转化为给定 N 个区间，选最少的点使每个区间内至少有一个点
- **贪心策略：**
 - 将所有区间按右端点从小到大排序
 - 选择第一个区间的右端点作为基站
 - 跳过所有包含该基站的区间，找到第一个未被覆盖的区间，选其右端点
- **时间复杂度：** $O(N \log N)$

```
struct Interval {
    double l, r;
};

bool cmp(Interval a, Interval b) {
    return a.r < b.r;
}

int solve_radar(int n, double d,
               vector<pair<double, double>> points) {
    vector<Interval> segs;
    for(auto& p : points) {
        if(p.second > d) return -1;
        double delta = sqrt(d*d - p.second*p.second);
        segs.push_back({p.first - delta,
                       p.first + delta});
    }
    sort(segs.begin(), segs.end(), cmp);

    int count = 0;
    double last_pos = -1e18;

    for(auto& s : segs) {
        if(s.l > last_pos) {
            count++;
            last_pos = s.r;
        }
    }
    return count;
}
```

解题步骤：

1. 对每个点，计算其在 x 轴上的覆盖区间，若 $y_i > d$ 则无解
2. 将所有区间按右端点从小到大排序
3. 初始化基站位置为极小值
4. 遍历排序后的区间，若当前区间左端点大于上一个基站位置，则在该区间右端点建立新基站
5. 继续遍历，直到处理完所有区间

5. 生产与存储计划

某公司有工厂和仓库。由于原材料等价格波动，工厂每个月的生产成本也会波动，令第 i 个月产品的单位生产成本为 c_i （该月生产一个产品的成本为 c_i ）。仓库储存产品的也有成本，假设每个月产品的单位储存成本为固定值 1（存储一个产品一个月的成本为 1）。令第 i 个月需要供应给客户的产品数量为 y_i ，仓库里的和生产的产品均可供应给客户。假设仓库的容量无限大，供应给客户剩余的产品可储存在仓库中。若已知 n 个月中各月的单位生产成本 c_i 、以及产品供应量 y_i ，设计一算法决策每个月的产品生产数量，使得 n 个月的总成本最低。

示例： $n = 3, c : 2, 5, 3, y : 2, 4, 5$ ，则 $x : 6, 0, 5$ 。即第 1 个月生产 6 个供应 2 个（代价 $2 \times 2 = 4$ ），储存 4 个供应给第 2 个月（代价 $(2 + 1) \times 4 = 12$ ），第 3 个月生产 5 个供应 5 个（代价 $3 \times 5 = 15$ ），使总成本 $4 + 12 + 15 = 31$ 最小。

- **贪心策略：**对于第 i 个月的需求 y_i ，寻找”最便宜的供应源”
- **供应源选择：**可以是当月生产（成本 c_i ），也可以是之前月份生产并存储（成本 $c_j + (i - j)$ ）
- **递推关系：**第 i 月的最优单位成本 $best_cost_i = \min(c_i, best_cost_{i-1} + 1)$
- **时间复杂度：** $O(N)$

```
void solve_production(int n,
                     vector<int> c,
                     vector<int> y) {
    vector<int> x(n, 0);
    long long total_cost = 0;

    pair<int, int> best_source = {c[0], 0};

    for(int i=0; i<n; ++i) {
        if (i > 0) {
            if (best_source.first + 1 < c[i]) {
                best_source.first += 1;
            } else {
                best_source = {c[i], i};
            }
        } else {
            best_source = {c[i], 0};
        }

        total_cost += (long long)best_source.first * y[i];
        x[best_source.second] += y[i];
    }

    cout << "最小总成本: " << total_cost << endl;
    cout << "生产计划 x: ";
    for(int val : x) cout << val << " ";
    cout << endl;
}
```

解题步骤：

1. 初始化最优供应源为第一个月，成本为 c_0
2. 对每个月份 i ：
 - 比较”延续之前存储”（成本 +1）与”当月生产”（成本 c_i ）
 - 选择更便宜的方案作为当前最优供应源
 - 将需求 y_i 的成本累加到总成本
 - 记录生产计划，需求由最优源所在月份生产
3. 输出最小总成本和生产计划

6. 黑白点连线

给定直线上 $2n$ 个点的序列 $P[1, 2, \dots, 2n]$ ，每个点 $P[i]$ 要么是白点要么是黑点，其中共有 n 个白点和 n 个黑点，相邻两个点之间距离均为 1，请设计一个算法将每个白点与一黑点相连，使得连线的总长度最小。

示例：图中有 4 个白点和 4 个黑点，以图中方式相连，连线总长度为 $1 + 1 + 1 + 5 = 8$ 。

- **贪心策略：顺序匹配**——第 1 个白点连第 1 个黑点，第 2 个白点连第 2 个黑点，以此类推
- **直观理解：**避免连线”交叉”，交叉的连线通常比不交叉的更长（三角形两边之和大于第三边）
- **时间复杂度：** $O(N \log N)$ （主要是排序）

```
int solve_connect(int n,
                 vector<int> whites,
                 vector<int> blacks) {
    sort(whites.begin(), whites.end());
    sort(blacks.begin(), blacks.end());

    int total_len = 0;
    for(int i=0; i<n; ++i) {
        total_len += abs(whites[i] - blacks[i]);
    }
    return total_len;
}
```

解题步骤：

1. 将所有白点按坐标从小到大排序
2. 将所有黑点按坐标从小到大排序
3. 依次将第 i 个白点与第 i 个黑点配对
4. 计算每对点的距离并累加
5. 返回总距离

正确性说明（交换论证）：

假设最优解中存在交叉配对，即存在白点 $w_i < w_j$ 和黑点 $b_k < b_l$ ，使得 w_i 连 b_l ， w_j 连 b_k 。则可交换为 w_i 连 b_k ， w_j 连 b_l ，新距离 $|w_i - b_k| + |w_j - b_l| \leq |w_i - b_l| + |w_j - b_k|$ ，因此不会更差。重复消除所有交叉后得到顺序匹配。

7. 活动选择问题

活动集合 $S = \{1, 2, \dots, n\}$ ，活动 i 有起始时间 s_i 和终止时间 f_i ($s_i \leq f_i$)，活动表示为 $x_i = [s_i, f_i]$ 。同一时间只有一个活动可以使用资源。活动 i 和 j 是相容的，当且仅当 $s_j \geq f_i$ 或 $s_i \geq f_j$ 。求 S 的最大相容子集。

- **贪心策略**：将活动按结束时间递增排序 ($f_1 \leq f_2 \leq \dots \leq f_n$)，每次选择结束时间最早且与已选活动相容的活动
- **时间复杂度**： $O(N \log N)$ (主要是排序时间)

```
struct Activity {
    int id, start, finish;
};

bool cmp(Activity a, Activity b) {
    return a.finish < b.finish; // 按结束时间排序
}

vector<int> greedyActivitySelector(
    vector<Activity>& activities) {
    sort(activities.begin(),
        activities.end(), cmp);

    vector<int> A;
    A.push_back(activities[0].id);

    for (int i = 1; i < activities.size(); i++) {
        // 如果当前活动的开始时间
        // >= 最后一个选中活动的结束时间
        if (activities[i].start >=
            activities[A.back()].finish) {
            A.push_back(activities[i].id);
        }
    }
    return A;
}
```

算法步骤：

1. 将所有活动按结束时间从小到大排序
2. 选择第一个活动（结束最早的活动）加入集合 A
3. 从第二个活动开始遍历，若当前活动与 A 中最后一个活动相容（即开始时间 $\geq$ 最后活动的结束时间），则加入 A
4. 返回相容活动集合 A

最优子结构性质：设活动按结束时间递增排序，若 A 是包含活动 1 的最优解，则 $A' = A - \{1\}$ 是 $S' = \{i \in S \mid s_i \geq f_1\}$ 的最优解。

贪心选择性：每次选择结束时间最早的相容活动，总能得到最优解。

8. 哈夫曼编码问题

给定字符集 $C = \{c_1, c_2, \dots, c_n\}$ 及其出现频率 $f = \{f_1, f_2, \dots, f_n\}$ ，构造 C 的最优前缀编码，使带权路径长度 $B(T) = \sum_{c \in C} f(c) \cdot d_T(c)$ 最小，其中 $d_T(c)$ 是字符 c 在二叉树 T 中的深度。

- **贪心策略**：每次选择频率最小的两个字符（或子树），合并为一棵新的子树，其频率为两者之和
- **数据结构**：使用优先队列（最小堆）维护待合并的节点
- **时间复杂度**： $O(N \log N)$

```
struct Node {
    int freq;
    char ch;
    Node *left, *right;

    Node(int f, char c = '$',
        Node* l = nullptr, Node* r = nullptr)
        : freq(f), ch(c), left(l), right(r) {}
};

struct Compare {
    bool operator()(Node* a, Node* b) {
        return a->freq > b->freq;
    }
};

Node* buildHuffmanTree(
    map<char, int>& freqMap) {
    priority_queue<Node*,
        vector<Node*>,
        Compare> minHeap;

    for (auto& p : freqMap) {
        minHeap.push(new Node(p.second, p.first));
    }

    while (minHeap.size() > 1) {
        Node* left = minHeap.top();
        minHeap.pop();

        Node* right = minHeap.top();
        minHeap.pop();

        Node* parent = new Node(
            left->freq + right->freq,
            '$', left, right);

        minHeap.push(parent);
    }
    return minHeap.top();
}
```

算法步骤：

1. 为每个字符创建一棵只包含根节点的二叉树，根节点权值为该字符频率，得到森林 $F = \{T_1, T_2, \dots, T_n\}$
2. 重复以下步骤，直到 F 中只剩一棵树：
 - 从 F 中选择根节点权值最小的两棵二叉树，作为左、右子树构造一棵新的二叉树
 - 新树的根节点权值等于其左右子树根节点权值之和
 - 从 F 中删除这两棵二叉树，并将新构造的二叉树加入 F
3. F 中剩下的最后一棵树即为哈夫曼树，左分支编码为 0，右分支编码为 1

9. 最小生成树问题 (Kruskal 算法)

给定带权无向连通图 $G = (V, E)$, 权函数 $W : E \rightarrow \mathbb{R}$, 求 G 的最小生成树, 即权值 $W(T) = \sum_{(u,v) \in T} W(u,v)$ 最小的生成树 $T = (V, T)$, 其中 $T \subseteq E$.

- **贪心策略:** 按权值从小到大选择边, 若该边的两个端点属于不同的连通分量, 则加入生成树; 否则舍弃
- **关键数据结构:** 并查集 (Union-Find) 用于快速判断两个顶点是否属于同一连通分量
- **时间复杂度:** $O(E \log E)$ 或 $O(E \log V)$

```
struct Edge {
    int u, v, weight;
    bool operator<(const Edge& other) const {
        return weight < other.weight; }
};

class UnionFind {
    vector<int> parent, rank;
public:
    UnionFind(int n) : parent(n), rank(n, 0) {
        for (int i = 0; i < n; i++)
            parent[i] = i; }
    int find(int x) {
        if (parent[x] != x)
            parent[x] = find(parent[x]);
        return parent[x]; }
    bool unionSets(int x, int y) {
        int px = find(x), py = find(y);
        if (px == py) return false;
        if (rank[px] < rank[py])
            parent[px] = py;
        else if (rank[px] > rank[py])
            parent[py] = px;
        else { parent[py] = px; rank[px]++; }
        return true; }
};

int kruskal(int n, vector<Edge>& edges) {
    sort(edges.begin(), edges.end());
    UnionFind uf(n);
    int totalWeight = 0, edgeCount = 0;
    for (auto& e : edges) {
        if (uf.unionSets(e.u, e.v)) {
            totalWeight += e.weight;
            if (++edgeCount == n - 1) break;
        }
    }
    return totalWeight;
}
```

算法步骤:

1. 初始化: 每个顶点自成一个连通分量
2. 将所有边按权值从小到大排序
3. 遍历排序后的边, 对于每条边 (u, v) :
 - 若 u 和 v 属于不同连通分量, 则加入最小生成树, 合并这两个连通分量
 - 否则, 舍弃该边 (会形成环)
4. 重复直到生成树包含 $V - 1$ 条边或所有边已处理

正确性证明:

- **最优子结构:** 设 T 是 G 的最小生成树, 包含子树 T_1 和 T_2 , 则 T_1 和 T_2 分别是各自子图的最小生成树
- **贪心选择性:** 设边 (u, v) 是当前权值最小且两端点分属不同连通分量的边, 则必存在一棵最小生成树包含 (u, v)

10. 单源最短路径问题 (Dijkstra 算法)

给定带权有向图 $G = (V, E)$, 每条边权值 ≥ 0 , 给定源点 v_0 , 求 v_0 到其他所有顶点的最短路径长度及路径。

- **贪心策略:** 维护已确定最短路径的顶点集合 S 和未确定的集合 $V - S$, 每次从 $V - S$ 中选择距离源点最近的顶点 u 加入 S , 然后松弛 u 的所有邻接边
- **松弛操作 (Relaxation):** 若 $dist[v] > dist[u] + W(u, v)$, 则更新 $dist[v] = dist[u] + W(u, v)$
- **时间复杂度:** $O(V^2)$ (简单实现) 或 $O((V + E) \log V)$ (使用优先队列)

```
const int INF = 1e9;
void dijkstra(int n, int src,
              vector<vector<pair<int, int>>& adj) {
    vector<int> dist(n, INF);
    vector<int> parent(n, -1);
    vector<bool> visited(n, false);
    dist[src] = 0;
    for (int i = 0; i < n; i++) {
        int u = -1, minDist = INF;
        for (int j = 0; j < n; j++) {
            if (!visited[j] && dist[j] < minDist) {
                u = j; minDist = dist[j]; }
        }
        if (u == -1) break;
        visited[u] = true;
        for (auto& edge : adj[u]) {
            int v = edge.first, weight = edge.second;
            if (!visited[v] && dist[u] + weight < dist[v]) {
                dist[v] = dist[u] + weight;
                parent[v] = u;
            }
        }
    }
    for (int i = 0; i < n; i++) {
        cout << "dist[" << i << "] = " << dist[i] << endl;
    }
}
```

算法步骤:

1. 初始化: $dist[v_0] = 0$, $dist[v] = \infty$ ($v \neq v_0$), $S = \emptyset$
2. 重复以下步骤, 直到 S 包含所有顶点或无法继续:
 - 从 $V - S$ 中选择 $dist[u]$ 最小的顶点 u
 - 将 u 加入 S
 - 对 u 的每个邻接顶点 $v \in V - S$, 执行松弛操作:

若 $dist[v] > dist[u] + W(u, v)$ 则 $dist[v] = dist[u] + W(u, v)$, $parent[v] = u$

3. 返回距离数组 $dist[]$ 和前驱数组 $parent[]$

路径还原: 通过 $parent[]$ 数组反向回溯可得到最短路径。例如, $parent[4] = 3, parent[3] = 2, parent[2] = 0$, 则源点到顶点 4 的路径为 $0 \rightarrow 2 \rightarrow 3 \rightarrow 4$ 。

正确性证明:

- **最优子结构:** 若 $P(s, d)$ 是 s 到 d 的最短路径, 则 P 的子路径 $P(u, v)$ 也是 u 到 v 的最短路径
- **贪心选择性:** 设 A 是已确定最短路径的顶点集, B 是未确定的。若 $dist[u] + W(u, v)$ 对所有 $u \in A, v \in B$ 最小, 则 $P + \{(u, v)\}$ 是 s 到 v 的最短路径